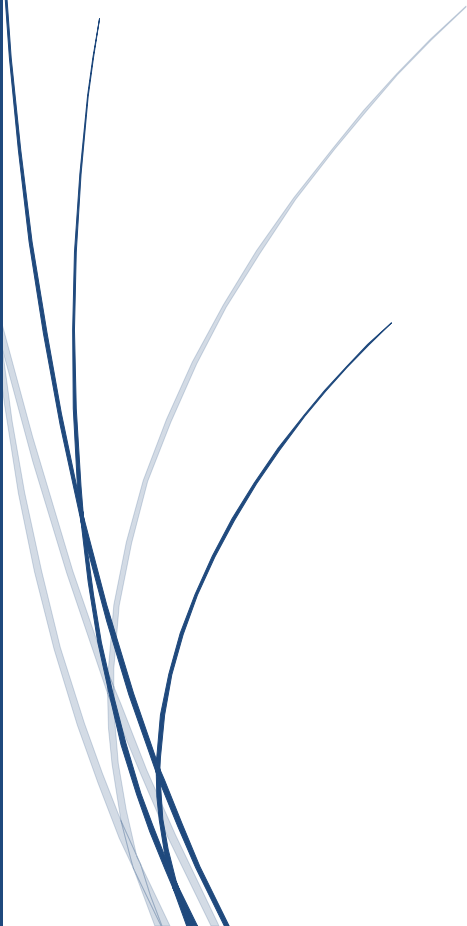


A thick dark blue vertical bar runs down the left side of the page. A blue arrow points to the right from this bar, containing the date.

19-2-2025

MEMORIA PRÁCTICA1

BASES DE DATOS 2

Several thin, curved lines in dark blue and light grey originate from the bottom left and sweep upwards and to the right.

ÁLVARO PÉREZ GREGORIO – 870097
MARIO RECIO LUEZA - 838583

Tabla de Contenido

Introducción	2
Objetivo	2
Motivación	2
Diseño BD.....	3
Nuestra Base de datos	3
Configuración en las acciones Borrar y Actualizar	3
Análisis Rendimiento.....	5
Inserción.....	5
Consultas	6
Optimización	9
Conclusiones	11
Bibliografía	12

Introducción

En esta práctica se nos pide diseñar e implementar una base de datos definiendo sus tablas, atributos y relaciones en un sistema gestor de bases de datos. Para ello, nosotros hemos elegido MariaDB, el cual está basado en MySQL. Además, a lo largo del trabajo deberemos realizar un análisis de rendimiento a la hora de la inserción de los datos (de nivel bajo, medio y alto los cuales veremos más adelante) así como de la ejecución de consultas SQL.

En nuestro caso hemos optado por realizar una base de datos relacionada con diferentes competiciones del fútbol universitario aunque esto lo explicaremos en detalle más adelante.

Objetivo

El principal objetivo de esta práctica es el de repasar y ampliar un poco los conocimientos que adquirimos en la asignatura de Bases de datos 1. Nos adentraremos en un entorno práctico SGBD como es MariaDB en este caso.

Se buscará no solo diseñar una base de datos funcional sino también explorar aspectos clave como la optimización de consultas y el análisis de rendimiento anteriormente comentado, a la vez que ver las diferencias entre los diferentes motores de almacenamiento que hay para gestionar las tablas de la base de datos.

Motivación

Elegimos MariaDB como SGBD para esta práctica ya que se nos presenta en la asignatura como un sistema compatible con MySQL con el que diseñar estructuras de datos robustas, esto nos llamó la atención además de que era un sistema que aún no habíamos utilizado.

Finalmente esta práctica nos da una oportunidad para aprender y formarnos mejor a la hora de realizar y manejar bases de datos en un futuro, ya que este gestor es muy común entre las empresas.

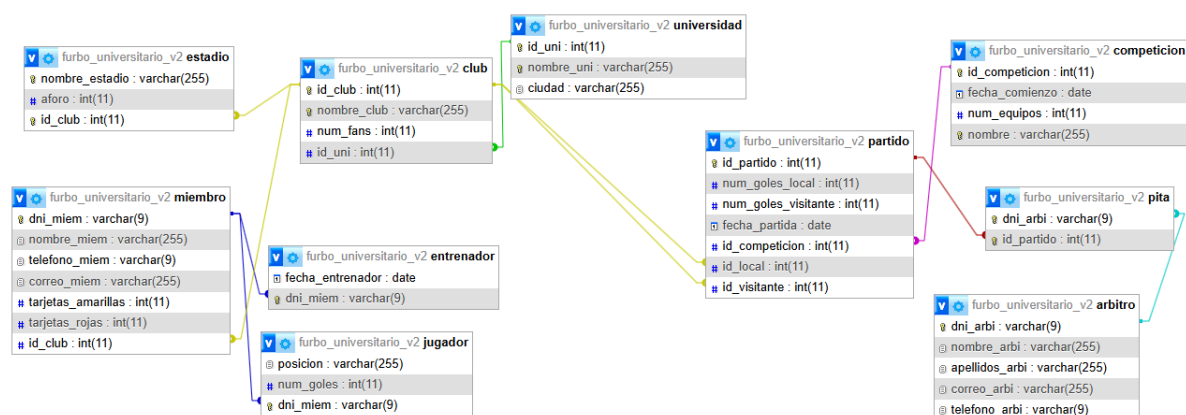
Diseño BD

Nuestra Base de datos

Para la elaboración de esta práctica hemos enfocado la temática en el fútbol para estar más familiarizados con qué datos trabajamos. Se llamará Furbo Universitario. La base de datos consistirá en tres entidades principales, Club, Miembro y Partido, donde los Clubes pertenecerán a una Universidad y tendrán varios Miembros los cuales cuentan con una generalización especificación total solapada así que podrán ser Jugadores o Entrenadores. El Club también dispondrá de un Estadio pero como se puede ver en el diseño tendrá la clave primaria de Club, es decir, será una entidad débil.

Por otro lado, Partido, competirán dos entidades de Club, pertenecerá a una Competición y lo “pitarán” uno o varios Árbitros. Por la relación Partido-Árbitro debemos crear otra tabla llamada Pita en consecuencia a ser una relación N-N.

A continuación podemos ver el diseño de las tablas en phpMyAdmin:



Configuración en las acciones Borrar y Actualizar

Usaremos la configuración CASCADE, tanto en las acciones borrar como actualizar, en la relación de *Club* con *Estadio*, ya que *Estadio* al ser entidad débil cada vez que se elimine un *Club* se eliminará su *Estadio* asociado y también convendría que al actualizarse los datos de *Club* se deberían actualizar en *Estadio*.

En la relación entre *Partido* y *Competición* también usaremos la configuración de CASCADE ya que si una *Competición* es borrada o actualizada, los partidos de esa competición deberían sufrir esos cambios también.

También entre *Club* y *Universidad* utilizaremos esta configuración ya que como venimos diciendo en los otros ejemplos si eliminamos una *Universidad* se eliminarían todos sus *Clubes* asociados. Pasaría lo mismo con la opción actualizar.

Por último entre *Miembro* y *Jugador* y *Entrenador* también la usaremos ya que un Jugador y un Entrenador dependerán del Miembro así que si este se actualiza o se elimina también ocurrirá para los que lo heredan.

Por otra parte usaremos la configuración CASCADE a la hora de actualizar y SET NULL a la hora de borrar para la relación entre *Club* y *Miembro* así al borrar un club no se borrarán los *Miembros* sino que simplemente se desvincularán del *Club*.

Análisis Rendimiento

Para el análisis de rendimiento hemos utilizado dos motores MySQL para ponerlos a prueba, en nuestro caso serán InnoDB y MyISAM.

Inserción

Para la inserción de los datos hemos seleccionado una cantidad arbitraria siguiendo el plan de tres tasas diferentes de datos (cantidades bajas, medias y altas) que mostramos en la siguiente tabla:

NºDatos	Pequeño	Medio	Alto
Universidad	100	1000	10000
Club	10000	40000	100000
Estadio	10000	40000	100000
Miembro	40000	300000	1000000
Jugador	20000	150000	500000
Entrenador	20000	150000	500000
Competcion	750	2000	6000
Partido	15000	80000	500000
Pita	30000	160000	1000000
Arbitro	15000	20000	100000
Total Datos	160850	943000	3816000

A la hora de insertarlos en la base hemos realizado un script para cada motor compatible con php y XAMPP para que genere datos aleatorios para cada tabla según la cantidad indicada. A continuación podemos ver los tiempos de inserción que se han tardado para cada tabla en InnoDB y en MyISAM:

Innodb	Pequeño	Medio	Alto	MyISAM	Pequeño	Medio	Alto
Universidad	0,0055	0,0019	0,151	Universidad	0,0011	0,006	0,066
Club	0,064	0,21	1,12	Club	0,067	0,255	0,846
Estadio	0,065	0,371	1,04	Estadio	0,053	0,203	0,623
Miembro	0,299	4,294	14,876	Miembro	0,264	2,745	9,893
Jugador	0,095	2,038	40,653	Jugador	0,099	0,823	2,236
Entrenador	0,11	1,111	34,543	Entrenador	0,087	0,659	3,251
Competcion	0,009	0,02	0,231	Competcion	0,006	0,014	0,782
Partido	0,163	2,296	20,764	Partido	0,076	0,549	3,813
Pita	0,495	2,745	189,361	Pita	0,151	1,187	8,498
Arbitro	0,037	0,282	1,529	Arbitro	0,038	0,251	0,861
Total Tiempo	1,3425	13,3689	304,268	Total Tiempo	0,8421	6,692	30,869

~Todos los tiempos se miden en segundos~

Lo primero que observamos es la diferencia de tiempos frente a la cantidad de datos insertados, podemos ver como aumentan los tiempos según aumentan las tasas de datos, es decir, cuantos más datos se tienen que añadir más tiempo conllevará la inserción.

Nos fijamos también a simple vista en la gran ventaja de tiempo que le saca el motor MyISAM al motor InnoDB. Esto se debe principalmente a que en MyISAM no se permiten las claves ajenas por lo que no tardará tanto en comprobar índices y en conectar tablas. Para responder a qué se debe tal diferencia lo explicaremos y representaremos más amplio en la siguiente tabla.

	InnoDB	MyISAM
Transacciones	Cada inserción se registra en el log	No usa transacciones
Bloqueo de datos	Bloquea fila por fila	Bloquea toda la tabla
Índices	Cada inserción actualiza el índice en el disco	Se actualizan en bloque

En MyISAM al no utilizar transacciones será mucho más rápido y al bloquear toda la tabla al mismo tiempo será mucho más eficiente para el tiempo en inserciones masivas.

Consultas

En cuanto a las consultas, las hemos ejecutado todas utilizando la tasa de datos más alta para así disponer de una mayor cantidad de información con la que trabajar. Esto nos permitirá obtener tiempos de ejecución más diferenciados entre ambos motores. Las consultas realizadas son las siguientes:

1.- La primera consulta mostrará los 10 clubes que han anotado más goles en sus partidos ganados durante el último año.

```
SELECT c.nombre_club AS club, SUM(p.num_goles_local + p.num_goles_visitante) AS total_goles
FROM partido p
JOIN club c ON c.id_club = p.id_local OR c.id_club = p.id_visitante
WHERE p.fecha_partida >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
AND ((p.num_goles_local > p.num_goles_visitante AND c.id_club = p.id_local)
OR (p.num_goles_visitante > p.num_goles_local AND c.id_club = p.id_visitante))
GROUP BY c.id_club
ORDER BY total_goles DESC
LIMIT 10;
```

Los tiempos de ejecución son 13,8759 segundos en **InnoDB** y 15,9807 segundos en **MyISAM**.

2.- Esta consulta muestra todos los partidos en los que un club en particular (en este caso “Club58”) ha jugado, indicando el rival, la fecha y el resultado, ordenados del más reciente al más antiguo.

```
SELECT p.id_partido, p.fecha_partida, cl_local.nombre_club AS equipo_local,
cl_visitante.nombre_club AS equipo_visitante, p.num_goles_local, p.num_goles_visitante
FROM partido p
JOIN club cl_local ON p.id_local = cl_local.id_club
JOIN club cl_visitante ON p.id_visitante = cl_visitante.id_club
WHERE cl_local.nombre_club = 'Club58' OR cl_visitante.nombre_club = 'Club58'
ORDER BY p.fecha_partida DESC;
```

Los tiempos de ejecución son 0,205 segundos en **InnoDB** y 0,0665 segundos en **MyISAM**.

3.- Esta consulta cuenta los partidos ganados por cada club (ya sea jugando como local o visitante) y los muestra en orden descendente, con el club más exitoso en la parte superior.

```
SELECT
    cl.id_club,
    cl.nombre_club,
    COUNT(p.id_partido) AS partidos_ganados
FROM partido p
JOIN club cl ON (p.id_local = cl.id_club AND p.num_goles_local > p.num_goles_visitante)
OR (p.id_visitante = cl.id_club AND p.num_goles_visitante > p.num_goles_local)
GROUP BY cl.id_club, cl.nombre_club
ORDER BY partidos_ganados DESC;
```

Los tiempos de ejecución son 13,7071 segundos en **InnoDB** y 16,3446 segundos en **MyISAM**.

4.- Esta consulta obtiene un listado de todos los partidos con detalles como la competición, los equipos, el resultado y los árbitros. Si un partido no tiene árbitros, se muestra “Sin árbitros”. Los resultados se muestran ordenados del más reciente al más antiguo.

```
SELECT p.id_partido, p.fecha_partida, c.nombre AS nombre_competicion, cl_local.nombre_club AS
equipo_local, cl_visitante.nombre_club AS equipo_visitante, p.num_goles_local,
p.num_goles_visitante, COALESCE(GROUP_CONCAT(DISTINCT CONCAT(a.nombre_arbi, ' ',
a.apellidos_arbi) SEPARATOR ' '), 'Sin árbitros') AS arbitros
FROM partido p
JOIN competicion c ON p.id_competicion = c.id_competicion
JOIN club cl_local ON p.id_local = cl_local.id_club
JOIN club cl_visitante ON p.id_visitante = cl_visitante.id_club
LEFT JOIN pitas pt ON p.id_partido = pt.id_partido -- Relación muchos a muchos
LEFT JOIN arbitro a ON pt.dni_arbi = a.dni_arbi -- Unir con árbitros
```

```
GROUP BY p.id_partido, p.fecha_partida, c.nombre, cl_local.nombre_club, cl_visitante.nombre_club,
p.num_goles_local, p.num_goles_visitante
ORDER BY p.fecha_partida DESC;
```

Los tiempos de ejecución son 17,1933 segundos en **InnoDB** y 20,9516 segundos en **MyISAM**.

Tras probar estas consultas vemos que los tiempos entre motores no son muy diferenciados pero sí que varían en cuanto a ser uno más mayor que el otro y viceversa. Es por eso que hemos investigado para que tipos de consultas funciona de una forma más eficiente cada motor.

Si necesitamos consultas rápidas de solo lectura, hacemos muchos SELECTs sin transacciones ni JOINS o necesitamos búsquedas eficientes FULLTEXT, **MyISAM** será la mejor opción mientras que por otro lado si tenemos muchas actualizaciones y concurrencia, se manejan transacciones y relaciones entre tablas y se necesita integridad referencial y rendimiento estable **InnoDB** será la opción más eficiente.

Para finalizar con este apartado expondremos una tabla de ampliación con más diferencias del rendimiento entre los motores a la hora de hacer diferentes tipos de consultas.

Operación	MyISAM	InnoDB
SELECT simple	✓ Más rápido	✗ Más lento
SELECT con JOINS	✗ Más lento	✓ Más rápido
SELECT con ORDER BY y LIMIT	✓ Más rápido	✗ Más lento
INSERT masivo	✓ Más rápido	✗ Más lento
INSERT con transacciones	✗ Sin soporte	✓ Soporte
UPDATE con muchas filas	✗ Más lento	✓ Más rápido
DELETE masivo	✗ Más lento	✓ Más rápido
Búsqueda FULLTEXT	✓ Soporte	✗ Más limitado
Consultas concurrentes	✗ Más lento	✓ Más rápido

Optimización

A la hora de insertar datos no hemos tenido ningún problema hasta tener que insertar más de 100000 datos, donde ya hemos tenido problemas tanto con phpMyAdmin como con Workbench y esto era debido a que las consultas de inserción no estaban optimizadas, lo que produce fallos en los programas, que se queden bloqueados o incluso que se sobrecargue el ordenador y no poder usarlo hasta que se procesase todo, esto daba a que el tiempo fuese excesivamente alto.

Para optimizar las consultas de inserción hemos probado varias cosas que a continuación vamos a explicar:

1. Inserciones individuales (más lenta)

Ejemplo:

```
INSERT INTO club (id_club, nombre_club, num_fans, id_uni)
VALUES (1, 'Club1', 100, 1);
```

```
INSERT INTO club (id_club, nombre_club, num_fans, id_uni)
VALUES (2, 'Club2', 200, 2);
```

Cada vez que ejecutas un INSERT, MySQL:

- Abre una transacción.
- Inserta el registro.
- Actualiza los índices.
- Confirma la transacción.
- Repite todo esto por cada fila.

Porque es tan lenta:

- Cada inserción es un proceso independiente.
- Cada INSERT genera escrituras y actualizaciones de índices por separado.
- Es ineficiente para grandes volúmenes de datos.

2. Inserción múltiple (más rápida)

Ejemplo:

```
INSERT INTO club (id_club, nombre_club, num_fans, id_uni)
VALUES (1, 'Club1', 100, 1), (2, 'Club2', 200, 2), (3, 'Club3', 300, 3);
```

Cómo funciona:

- Agrupa varios valores en un solo INSERT.
- Solo abre y cierra la transacción una vez.
- Actualiza los índices de golpe, no por cada fila.

Por qué es más rápida:

- Reduce las operaciones de disco y el número de transacciones.
- Más eficiente en la actualización de índices.

En nuestro caso:

PhpMyAdmin limita el número de filas que puedes insertar en un solo INSERT. Si intentas meter 100,000 filas de golpe, se produce un error o se quedaba pillado el programa, por lo que conviene hacerlo en bloques de 10,000 o 20,000.

3. Carga masiva con LOAD DATA INFILE (la más rápida)

Ejemplo:

```
LOAD DATA INFILE 'ruta/datos.csv'
INTO TABLE club
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
(id_club, nombre_club, num_fans, id_uni);
```

Cómo funciona:

- Lee los datos directamente desde un archivo.
- Inserta todas las filas de una sola vez.
- Es el método que usa MySQL internamente para importar grandes cantidades de datos.

Por qué es la más rápida:

- Evita pasar por el protocolo de cliente-servidor fila por fila.
- Usa optimizaciones internas de MySQL.
- Evita múltiples comprobaciones y validaciones durante la carga.

Otras optimizaciones realizadas:

Técnica	Mejora Esperada
Desactivar claves(MyISAM)	3x - 5x
Desactivar el autocommit(InnoDB)	2x - 4x
Dividir en bloques de 10.000 - 20.000 filas	Evita fallos y es más fluido

Conclusiones

A lo largo de esta práctica, hemos diseñado e implementado una base de datos en MariaDB con la temática de fútbol universitario incidiendo en aspectos clave como el rendimiento y la optimización del ejercicio. La comparación entre los dos motores de almacenamiento nos permitió analizar sus ventajas y desventajas en base a la inserción de datos y consultas, lo que nos dio como resultado que MyISAM es más eficiente en operaciones de solo lectura y de carga masiva de datos, mientras que InnoDB será mejor en el desempeño de consultas con transacciones y relaciones complejas.

En el análisis de rendimiento, vemos como el volumen de datos influye en los tiempos de ejecución así cuanto mayor sea la tasa de datos mayor será el tiempo de inserción. El uso de inserciones múltiples y la carga masiva con *LOAD DATA INFILE* pueden optimizar significativamente los tiempos de procesamiento.

Para acabar cabe mencionar que esta práctica nos ha permitido afianzar conocimientos sobre bases de datos a la hora de entender mejor el análisis de rendimiento y la optimización además de adentrarnos en un SGBD nuevo para nosotros que es MariaDB.

Bibliografía

- *ChatGPT*. (n.d.). <https://chatgpt.com/>
- *DeepSeek*. (n.d.). Into the Unknown. <https://chat.deepseek.com/>
- *GitHub Copilot · Your AI pair programmer*. (2025). GitHub. <https://github.com/features/copilot>
- Mauricio Sevilla Britto. (2021, October 19). *Cómo hacer un Insert Masivo en MySQL* [Video]. YouTube. <https://www.youtube.com/watch?v=Tj-NDtBo0OU>
- Arsys. (2024, July 6). *¿MyISAM o InnoDB? Elige tu motor de almacenamiento MySQL*. <https://www.arsys.es/blog/myisam-o-innodb-elige-tu-motor-de-almacenamiento-mysql>
- Agrawal, V., & Agrawal, V. (2024, November 12). *MyISAM vs InnoDB: 7 Critical Differences*. Learn | Hevo. <https://hevodata.com/learn/myisam-vs-innodb/>